Emotet Data Construction and Labeling

Emotet network traffic was obtained from the Unit42 Wireshark Emotet repository. Four infection traces (Examples 1–4) were initially used to construct behavioural flow windows for my model's development and internal validation. These PCAP files provide controlled infection scenarios with documented compromised internal hosts. This source of data came with a key limitation.

The Unit42 repository did not include benign internal hosts, so clean background traffic was sourced separately from the Stratosphere Laboratory Normal Capture datasets. These captures represent non-infected internal activity and were used to construct the Normal class.

Labeling was based on confirmed infected internal hosts documented in the repository and verified via Zeek connection logs.

Example	Infected Internal Host
Example - 1	10.1.6.206
Example - 2	10.1.5.101
Example - 3	10.1.4.205
Example - 4	172.16.1.101

Clean Internal Hosts:

- normal_2017_04_30 – 10.0.2.15
- normal_2017_04_25 – 10.0.2.15

Malicious behavioural windows derived from confirmed infected hosts were labeled as Emotet, while windows extracted from clean captures were labeled as Normal. This host-based labeling ensures that class assignments are grounded in documented infection traces rather than heuristic assumptions.

Addressing Class Imbalance

Behavioural malware datasets are inherently imbalanced due to short infection durations, limited number of available PCAP captures, and high prevalence of benign background traffic. To mitigate this limitation, an additional infection trace (Example 5) was incorporated to expand the available Emotet behavioural windows. Subsequently, a balanced dataset was constructed with 555 samples per class (Normal, SQLi, Emotet). This controlled balancing ensures that performance metrics reflect class separability rather than majority-class dominance.

Multi Stage Detection Framework

My model consists of two datasets; SQLi dataset with payload-based features, and an Emotet dataset with network behavioural features. I developed a unified dataset to merge both into one structured table in order for my model to classify input as normal, SQLi, or Emotet. I built a unified multi-stage detection framework.

I chose a multi-class model over two separate binary models. This forces my model to learn: SQLi vs Normal, Emotet vs Normal, and SQLi vs Emotet. My model learns inter-attack distinctions, not just attack vs benign distinctions:

- 0 → Normal
- 1 → SQLi
- 2 → Emotet

This ensures my model understands structural differences between attack types, not just anomaly detection. My model's classifier can structurally distinguish between attack paradigms (lexical payload manipulation vs behavioural network anomalies)

Feature Schema Control — master_features_unified.json

Training and inference must use identical feature order, and column ordering in pandas is not guaranteed. Future dashboard predictions must align features correctly, and reproducibility requires schema freezing. If my feature list is NOT saved this way, my model may silently break, and result become unreproducible.

Stratified Split Justification

Stratified split is like a fairness constraint on my evaluation. Again, I used a stratified split to ensure the class proportions are preserved in both train and test. This is crucial in my case because my SQLi data takes up a major portion of the dataset, and through a stratified split, even my test set will still contain Emotet data.

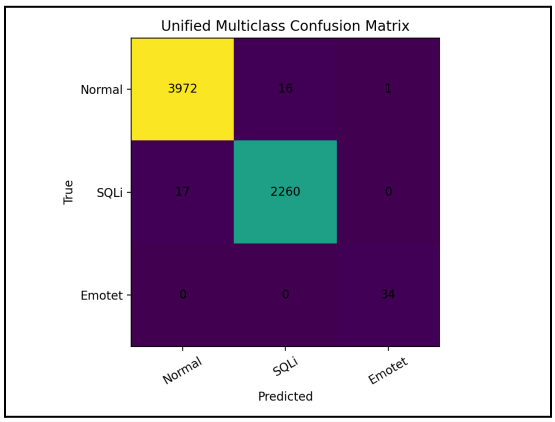
If I didn't use a stratified split, I risk having a test set with zero Emotet samples, or a training set with too few Emotet samples. As a result, my Emotet metrics become meaningless, and the ROC/PR curves for Emotet can't be computed properly.

My dataset is class-imbalanced, so if I train it normally, the model is incentivized to optimize overall accuracy by mainly getting the majority classes right. This means my model may look great while basically ignoring the minority class, here being Emotet. **class_weight="balanced"** will increase the penalty for mistakes on minority classes. This tells my model "Misclassifying Emotet is more costly than misclassifying Normal, because Emotet is rare." Basically, this function will make the trees in the forest consider splits that improve Emotet classification rather than sacrificing it for majority accuracy.

Multi-Class Evaluation

Test set size: 6300
Class Distribution:

- Normal: 3989
- SQLi: 2277
- Emotet: 34



True Normal = 3989, with 3972 labeled correctly, 16 misclassified as SQLi, and 1 misclassified as Emotet.

True SQLi = 2277, with 2260 correctly labeled, 17 misclassified as Normal, and 1 misclassified as Emotet.

True Emotet = 34, with 34 correct, 0 misclassified, and 0 FP from other classes.

The 17 misclassified as Normal is due to the fact that SQLi features are lexical and can sometimes overlap with benign queries containing special characters, longer strings, etc… so minor boundary overlap is expected.

There is no confusion or misclassification between SQLi and Emoetet, this means the model learned the SQLi feature space, and the Emotet feature space as two structurally distinct clusters.

Interpretation Per Class:

Emotet (2): Recall = 1.0, no attacks were missed
Precision: slightly less than 1 due to 1 false positive
F1: extremely high.
➔ The model detects all Emotet windows while maintaining an extremely low false alarm rate.

SQLi (1): high precision and recall, with a small number of borderline misclassifications. This suggests the lexical features are strong discriminators,

Normal (0): high precision and recall, crucial to have such characteristics because low precision would be a result of many false alarms (FPs).

These results show a clear separation between lexical SQLi features and behavioural Emotet features. However, my test set only contained 34 Emotet samples, and performance is strong but based on limited minority evaluation. To overcome this, group-holdout validation will test robustness further.

Macro-F1 vs Weighted-F1 — unified_classification_report.csv

Macro-F1 is used to treat all classes equally, regardless of how many samples they have, by averaging the F1 score of all classes. Even though Emotet has only 34 samples in the test set, Macro-F1 gives it equal importance.

- Macro Average, F1 = 0.991333
- Weighted Average, F1 = 0.994604
- Accuracy = 0.994603

Weighted F1 weights (here it's 0.994604) each class by its frequency, in my test set Emotet barely affects weighted F1, so this score is driven by dominant classes:

- Normal = 3989 samples
- SQLi = 2277 samples
- Emotet = 34 samples

This score reflects strong overall performance driven by dominant Normal and SQLi classes.

One-vs-Rest ROC-AUC — unified_auc_pr.csv

- Normal ROC-AUC = 0.999711
- SQLi ROC-AUC = 0.999725
- Emotet ROC-AUC = 0.999986

This measures ranking quality, evaluating the entire probability curve while disregarding the 0.5 threshold (explained in deeper detail in previous sections). This value indicates excellent ranking ability across probability thresholds.

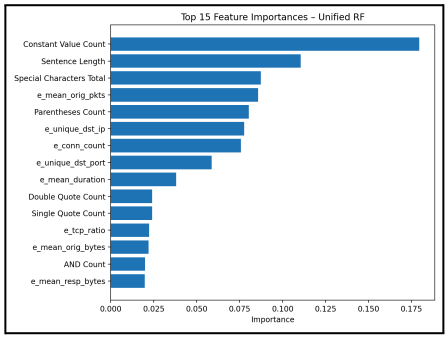
PR-AUC — unified_auc_pr.csv

The Emotet PR-AUC is 0.997403, and looking at this value in my imbalanced dataset is crucial because ROC may look excellent, even when precision collapses under rare positives. PR-AUC focuses on precision vs recall tradeoff, which is stricter when positive class is rare (in my case, Emotet with 34 samples). Through this, high recall is maintained without sacrificing precision, making PR-AUC the most honest metric.

RF Feature Importance

feature_importances_ is impurity based importance; MDI, Mean Decrease in Impurity. This means that:

- Each tree splits tries to reduce impurity (Gini, as previously mentioned)
- If a feature frequently produces large impurity reductions across trees, it gets higher importance.



Unified RF – Feature Importance

I covered feature importance in my other reports for just SQLi, this is the feature importance graph for the new unified RF model.

My dataset contains both SQLi lexical, text-derived counts data, and behavioural Emotet traffic from network statistics. The top 15 contain both types because my model isn't solely an SQLi vs Normal model, or Emotet vs Normal. It's using both feature families to classify.

<u>SQLi-Driving Features</u>: reflect structural properties of injected SQL-like strings and boolean logic manipulation, allowing the model to separate SQLi requests from benign input.

<u>Emotet-Driving Features</u>: capture network flow regularities and endpoint diversity associated with botnet-style behaviour, enabling separation of Emotet windows from benign background traffic.

The model is learning two distinct detections paradigms simultaneously:

- ➔ Application-layer attack signals (SQLi lexical structure)
- ➔ Network-layer behavioural signals (Emotet communication statistics)

Limitations:

<u>Limitation in Emotet: Minority-class influence may be underrepresented</u>. My dataset contains 31,000 total rows, with only 173 Emotet in the full dataset, and just 34 in the test set. Even though I used class_weight="balanced", impurity-based importance is still computed based on overall impurity reduction across all nodes. Since the normal and SQLi data dominates numerically, most splits are driven by separating those two large classes. It's possible that some Emotet-specific features are slightly under ranked because they only meaningfully reduce impurity in a small subset of samples.

Limitation in Emotet Features: Bias toward continuous behavioural features, in my list of top 15, several continuous numerical features appear, like e_mean_orig_pkts and e_mean_duration. These continuous variables allow many possible split thresholds (>3.5, >4.2). In contrast, lexical features like AND Count and Parentheses Count often have small integer ranges (0-5, 0-10). Since behavioural features (Emotet features) are continuous, the tree has more opportunities to find impurity-reducing splits with them. So they may appear slightly more “important” simply because they gave the tree more flexibility during training. It does not imply that they’re more meaningful, it just means they had more chances to help split.

Limitation in SQLi: Correlated lexical features split importance, many of the SQLi features are not independent, such as Single Quote Count, Double Quote Count, AND Count, etc… This means that if Single Quote Count increases, Special Characters Total often increases. These features tie together, if two features encode overlapping SQL injection signals, the forest may split importance between them. This means that none of these features individually appear ‘dominant’, but together, they represent strong injection structure. As a result, importance ranking underestimates feature family influence and over-focuses on individual variables.

Importance is global, not instance-level. My feature importance findings just says “these features were useful overall”, however, they do not explain which specific features drove the prediction for an individual sample. In order for my model to be able to do that, SHAP values or local explanations can help overcome this limitation.